

This command starts the consumer that connects to the Kafka on port number 9092 on the local machine. It registers for reading the messages on *topic-1*. Because of the last option on the command line, the consumer receives all the messages on the chosen topic from the beginning.

Since the producer and consumer are connecting to the same broker and referring the same topic, the consumer receives and displays the messages on its terminal.

Now, let's use Kafka in the context of a practical application.

Case study

ABC is a hypothetical bus transport company, which has a fleet of passenger buses that ply between different cities across the country. Since ABC wants to track each bus in real-time for improving the quality of its operations, it comes up with a solution around Apache Kafka.

ABC first equips all its buses with devices to track their location. An operations centre is set up with Apache Kafka, to receive the location updates from each of the hundreds of buses. A dashboard is developed to display the current status of all the buses at any point in time. Figure 5 represents this architecture.

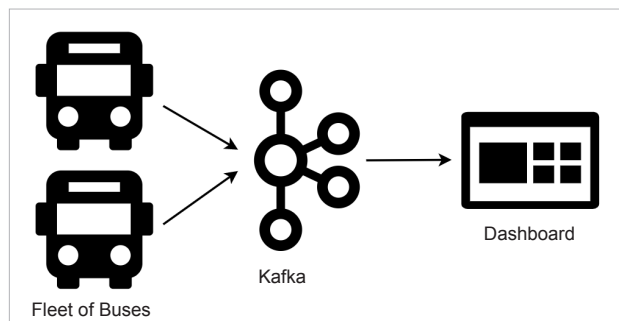


Figure 5: Kafka based architecture

In this architecture, the devices on the buses act as the message producers. They send their current location to Kafka on the topic *abc-bus-location*, periodically. For processing the messages from different buses, ABC chooses to use the trip code as the key. For example, if the bus from Bengaluru to Hubballi runs with the trip code BLRHBL003, then *BLRHBL003* becomes the key for all the messages from that specific bus during that specific trip.

The dashboard application acts as the message consumer. It registers with the broker against the same topic *abc-bus-location*. Consequently, the topic becomes the virtual channel between the producers (buses) and the consumer (dashboard).

The devices on the buses never expect any response from the dashboard application. In fact, none of them is even aware of the presence of the others. This architecture enables non-blocking communication between hundreds of buses and the central office.

Implementation

Let's assume that ABC wants to create three partitions for maintaining the location updates. Since the development environment has only one broker, the replication factor should be set to one.

The following command creates the topic accordingly:

```
./bin/kafka-topics.sh --create --topic abc-bus-location
--zookeeper localhost:2181 --partitions 3 --replication-
factor 1
```

The producer and consumer applications can be written in multiple languages like Java, Scala, Python, JavaScript, and a host of others. The code in the following sections provides a peek into the way they are written in Java.

Java producer: The Fleet class simulates the Kafka producer applications running on six buses of ABC. It sends location updates on *abc-bus-location* to the specified broker. Please note that the topic name, message keys, message body, and broker address are hard-coded only for simplicity.

```
public class Fleet {
    public static void main(String[] args) throws Exception
    {
        String broker = "localhost:9092";
        Properties props = new Properties();
        props.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_
CONFIG, broker);
        props.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_
CONFIG,
            StringSerializer.class.getName());
        props.setProperty(ProducerConfig.VALUE_SERIALIZER_
CLASS_CONFIG,
            StringSerializer.class.getName());

        Producer<String, String> producer = new
KafkaProducer<String, String>(props);
        String topic = "abc-bus-location";
        Map<String, String> locations = new HashMap<>();
        locations.put("BLRHBL001", "13.071362, 77.461906");
        locations.put("BLRHBL002", "14.399654, 76.045834");
        locations.put("BLRHBL003", "15.183959, 75.137622");
        locations.put("BLRHBL004", "13.659576, 76.944675");
        locations.put("BLRHBL005", "12.981337, 77.596181");
        locations.put("BLRHBL006", "13.024843, 77.546983");

        IntStream.range(0, 10).forEach(i -> {
            for (String trip : locations.keySet()) {
                ProducerRecord<String, String> record
                    = new ProducerRecord<String, String>(
                        topic, trip, locations.get(trip));
                producer.send(record);
            }
        });
    }
}
```